

Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 01.

알고리즘과 문제해결

경북대학교 배준현 교수
(joonion@knu.ac.kr)



알고리즘을 설계하는 두 가지 중요한 기준

1. 알고리즘의 **정확성**: *correctness*
주어진 문제의 **모든 입력 사례**에 대해
반드시 **올바른 해답**을 찾을 수 있음을 **증명**해야 한다.
2. 알고리즘의 **효율성**: *efficiency*
알고리즘이 주어진 문제를 해결하는 데
필요한 자원(시간, 공간)의 양을 **평가**해야 한다.



알고리즘의 정확성 증명

수학적 귀납법: *mathematical induction*

1. 기초 단계: 가장 작은 입력 값에 대해 알고리즘이 정확함을 증명한다.
2. 귀납 단계: 임의의 크기 k 에 대해 알고리즘이 정확하다고 가정하고, 크기 $k + 1$ 에 대해서도 올바르게 동작함을 증명한다.

루프 불변식: *loop invariant*

1. 초기화: 루프가 시작될 때 불변식이 참임을 증명한다.
2. 유지: 반복문의 각 반복 단계가 시작될 때 불변식이 참임을 증명한다.
3. 종료: 반복문이 종료될 때 불변식이 여전히 참임을 증명한다.

따라서, 알고리즘은 올바른 결과에 도달할 것임을 증명한다.

기타: 반례에 의한 증명, 직접 증명, 대우 증명, etc.



다음 알고리즘의 정확성을 증명하시오.

1부터 n 까지의 자연수의 합을 반환한다.

ALGORITHM: `sum1`(n : positive integer):

$s \leftarrow 0$;

for i from 1 to n :

$s \leftarrow s + i$;

return s ;

1. 수학적 귀납법을 이용하여 증명하시오.
2. 루프 불변식을 이용하여 증명하시오.



다음 알고리즘의 정확성을 증명하시오.

1부터 n까지의 자연수의 합을 반환한다.

ALGORITHM: `sum2`(`n`: positive integer):

return $\frac{n \times (n+1)}{2}$

1. 가우스의 방식을 이용하여 직접 증명하시오.
2. `sum1`, `sum2` 알고리즘(*pseudo-code*)을 프로그래밍 언어(C/C++ or Python)로 구현(*implementation*)할 수 있는가?



다음 알고리즘이 **유한성**을 위반하고 있음을 증명하시오.

```
# 양의 정수 n에 대해, 아래의 과정을 반복한 수열을 반환한다.  
#   n이 1이면 종료한다.  
#   n이 짝수이면 2로 나누고, 홀수이면 3을 곱하고 1을 뺀다.  
def weird(n):  
    if n == 1:  
        return [1]  
    elif n % 2 == 0:  
        return [n] + weird(n // 2)  
    else:  
        return [n] + weird(n * 3 - 1)
```



다음 알고리즘이 **유한성**을 위반하지 않음을 증명하시오.

양의 정수 n 의 콜라츠 수열을 반환한다.

```
def collatz(n):  
    if n == 1:  
        return [1]  
    elif n % 2 == 0:  
        return [n] + collatz(n // 2)  
    else:  
        return [n] + collatz(n * 3 + 1)
```



알고리즘의 효율성 분석

복잡도 분석: *complexity analysis*

알고리즘이 실행할 때 필요한 자원(시간, 공간)의 **효율성**을 평가하는 과정

시간 복잡도 함수: *input size*에 따른 *basic operation*의 수행 횟수에 대한 함수

점근적 분석: *asymptotic analysis*

입력의 크기가 매우 커질 때 알고리즘의 성능이 어떻게 변하는가를 평가

빅오 표기법: *Big-O notation*

입력 크기에 따라 수행 시간의 성장 속도가 달라지는 상한(*upper bound*)을 평가



다음과 같은 시간 복잡도를 가진 알고리즘의 성능을 빅오 표기법으로 표현하시오.

- $T(n) = 5$
- $T(n) = 3n + 2$
- $T(n) = \frac{n \times (n+1)}{2}$
- $T(n) = 7n \log_2 n + \log_2 n^7$
- $T(n) = n^{100} + \left(\frac{3}{2}\right)^n$
- $T(n) = n^{100} + \left(\frac{3}{2}\right)^n + (n - 2^{100})!$



계산 이론과 문제의 난이도

계산 이론: *computational theory*

알고리즘의 시간 복잡도가 아니라, 문제 자체의 시간 복잡도를 연구

선택 문제(selection problem)의 하한: $\Omega(n)$

비교에 기반한 정렬 문제(sorting problem)의 하한: $\Omega(n \log n)$

주로 문제의 하한(*lower bound*)
오메가(Ω) 표기법으로 표시

문제의 다루기 어려운 정도: *intractability*

풀기 쉬운 문제: 다항 시간 알고리즘이 존재하는 문제

풀기 어려운 문제: 다항 시간 알고리즘이 존재하지 않는 문제



Lec. 01. 알고리즘과 문제해결

계산 가능한 문제의 복잡도 클래스

해답을 찾는 다항 시간 알고리즘을 **아무도** 찾지 못했지만,
다항 시간 알고리즘이 존재할 수 없음을 **아무도** 증명하지 못한 문제

P-문제: 다항 시간(*polynomial-time*) 알고리즘이 알려진 문제

Shortest Path Finding, *Prime Determination* (A.K.S. algorithm), etc.

NP-문제: *Nondeterministic Polynomial*

다항 시간에 검증은 가능하지만, 다항 시간에 해결할 수 있을지 여부는 **결정적**이지 않은 문제

NP-완전: *NP-Complete*

NP-문제들 중에서 다항 시간에 환원이 가능한 결정 문제들의 집합 (NP-집합에 속함)

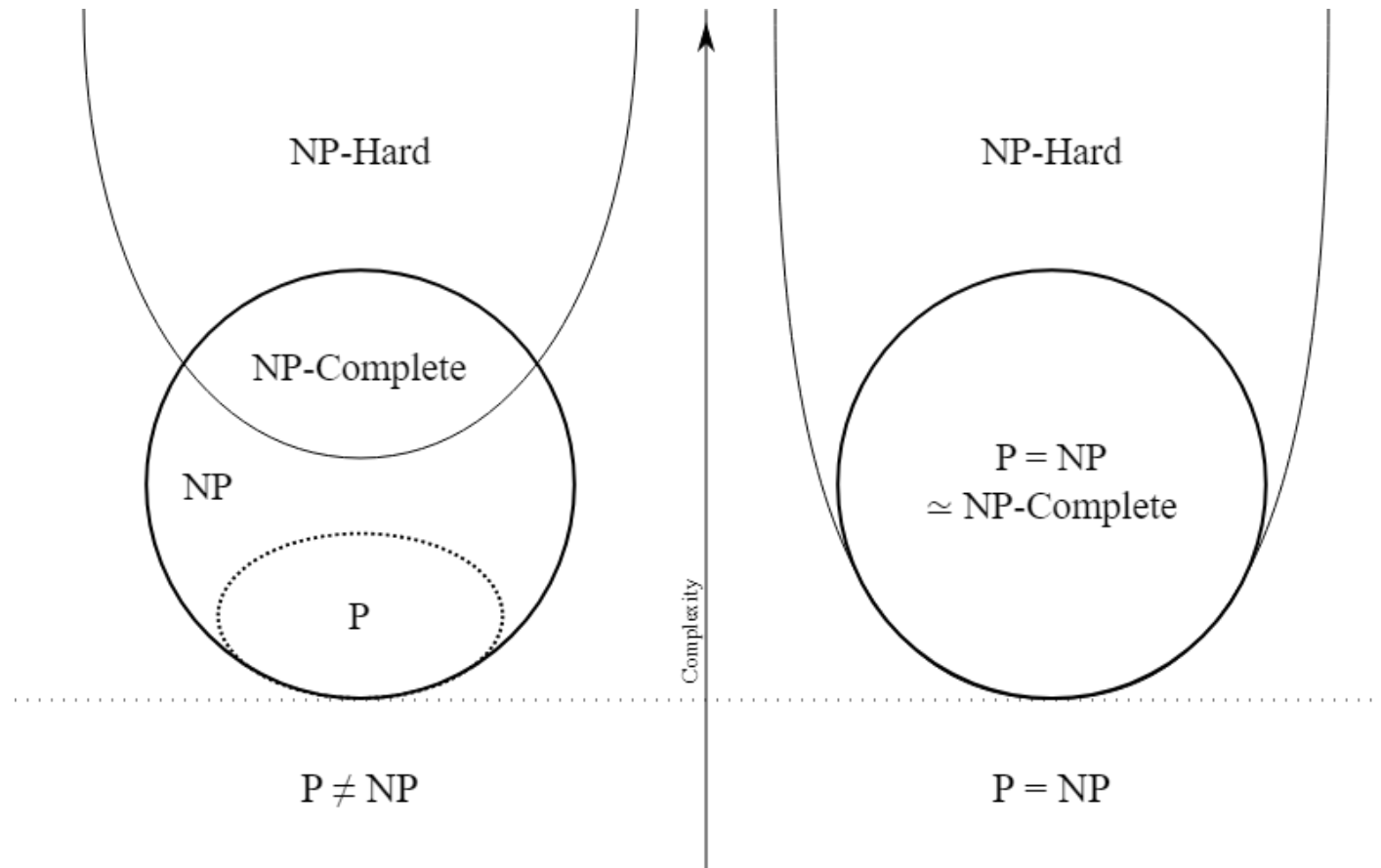
NP-난해: *NP-Hard*

NP-완전 문제보다 더 어려울 수 있는 문제들의 집합 (NP-집합에 속하지 않을 수도 있음)



Lec. 01. 알고리즘과 문제해결

P-NP 문제: $P = NP$? or $P \neq NP$?





AL2.1.1: 록 페스티벌 (ID: FESTIVAL)

문제

커다란 공연장을 빌려서 록 페스티벌을 개최하려고 합니다. 이 페스티벌은 여러 날 동안 진행되며, 하루에 한 팀의 밴드가 공연장에서 콘서트를 하게 됩니다. 전체 밴드를 몇 팀 섭외할 지는 아직 결정하지 않았지만, 페스티벌의 간판 스타인 L 개의 팀은 이미 섭외가 끝난 상태입니다. 따라서 페스티벌은 **최소 L 일 이상** 진행하게 됩니다.

이번에 사용할 공연장은 하루 빌리는 데 드는 비용이 매일 매일 다릅니다. 때문에 공연 일정을 잘 정해서 공연장 대여 비용을 줄이려고 합니다. 앞으로 **N 일간의 공연장 대여 비용**을 알고 있다고 합시다. 이 중 **L 일 이상을 연속해서 대여**하되, **공연장을 하루 빌리는 데 드는 평균 비용을 최소화**하려면 어떻게 공연장을 빌려야 할까요?



Lec. 01. 알고리즘과 문제해결

예를 들어 앞으로 6일간 공연장을 빌리는 데 드는 비용이 각 $\{3, 1, 2, 3, 1, 2\}$ 라고 합시다. 이미 세 팀을 섭외했다고 하면, 3일 대신 4일 동안 공연을 진행해서 평균 비용을 더 저렴하게 할 수 있습니다. 3일 동안의 평균 대여 비용을 최소화하는 방법은 2일째부터 4일째까지 공연장을 대여하는 것인데, 이 때 하루 평균 $(1+2+3)/3 = 2$ 의 비용이 듭니다. 반면 2일째부터 5일째까지 공연장을 대여하면 평균 비용이 $(1+2+3+1)/4 = 7/4$ 밖에 되지 않습니다.



Lec. 01. 알고리즘과 문제해결

입력

입력의 첫 줄에는 테스트 케이스의 수 C ($C \leq 100$)가 주어집니다. 각 테스트 케이스의 첫 줄에는 공연장을 대여할 수 있는 날들의 수 N ($1 \leq N \leq 1000$)과 이미 섭외한 공연 팀의 수 L ($1 \leq L \leq 1000$, $L \leq N$)이 주어집니다. 그 다음 줄에는 N 개의 숫자로 공연장 대여 비용이 날짜별로 주어집니다. 모든 비용은 100 이하의 자연수입니다.

출력

입력에 주어지는 각 테스트 케이스마다 한 줄에 최소의 평균 대여 비용을 출력합니다. 10^{-7} 이하의 절대/상대 오차가 있는 답은 정답 처리됩니다.



Lec. 01. 알고리즘과 문제해결

예제 입력

```
2
6 3
1 2 3 1 2 3
6 2
1 2 3 1 2 3
```

예제 출력

```
1.75
1.5
```



Lec. 01. 알고리즘과 문제해결

1단계: 문제를 읽고 이해한다.

$N = 6$: 전체 공연일정이 6일

$L = 3$: 최소 대여일정이 3일

$cost = [1, 2, 3, 1, 2, 3]$: 각 공연일별 대여 비용

$$\text{평균 대여 비용} = \frac{\text{공연일별 대여 비용의 합}}{\text{대여 일수}}$$

최소 평균 대여 비용: 가능한 대여 방법 중에서 가장 작은 평균 대여 비용

(계약 조건) 최소 L 일 이상은 대여해야 하고, 반드시 연속적으로 대여해야 한다.



Lec. 01. 알고리즘과 문제해결

2단계: **어떻게 풀 지 계획을 세운다.** (문제의 **추상화**와 **재정의**)

전체 대여 비용: $cost = [c_0, c_1, c_2, \dots, c_{n-1}]$

대여일(i to j)의 비용: $cost_{ij} = [c_i, c_{i+1}, c_{i+2}, \dots, c_j]$

평균 대여 비용: $avg_{ij} = \frac{sum(cost_{ij})}{j-i+1}$

최소 평균 대여 비용: $\min(avg_{ij})$ for all i, j where $j - i + 1 \geq L$



$N = 6, L = 3, cost = [1, 2, 3, 1, 2, 3]$

1	2	3	1	2	3
---	---	---	---	---	---

i j

$tot_cost = 1 + 2 + 3 = 6,$
 $avg_cost = 6/3 = 2$

1	2	3	1	2	3
---	---	---	---	---	---

i j

$tot_cost = 1 + 2 + 3 + 1 = 7,$
 $avg_cost = 7/4 = 1.75$

1	2	3	1	2	3
---	---	---	---	---	---

i j

$tot_cost = 1 + 2 + 3 + 1 + 2 = 9,$
 $avg_cost = 9/5 = 2.25$

1	2	3	1	2	3
---	---	---	---	---	---

i j

$tot_cost = 1 + 2 + 3 + 1 + 2 + 3 = 12,$
 $avg_cost = 12/6 = 2$



Lec. 01. 알고리즘과 문제해결

1	2	3	1	2	3	$tot_cost = 6, avg_cost = 6/3 = 2$
	i		j			
1	2	3	1	2	3	$tot_cost = 8, avg_cost = 8/4 = 2$
	i			j		
1	2	3	1	2	3	$tot_cost = 11, avg_cost = 11/5 = 2.2$
	i				j	
1	2	3	1	2	3	$tot_cost = 6, avg_cost = 6/3 = 2$
		i		j		
1	2	3	1	2	3	$tot_cost = 9, avg_cost = 9/4 = 2.5$
			i		j	
1	2	3	1	2	3	$tot_cost = 6, avg_cost = 6/3 = 2$
				i	j	

$min_cost = 1.75$



Lec. 01. 알고리즘과 문제해결

3단계: 계획을 수행해서 문제를 해결한다. (알고리즘의 설계와 구현)

ALGORITHM: FESTIVAL

INPUT: $N, L, cost$

OUTPUT: min_cost (최소 평균 대여 비용)

1. $min_cost \leftarrow \infty$: 최소 평균 대여 비용을 아주 큰 값으로 초기화

2. for i from 0 to $N - 1$: 가능한 모든 대여 시작일 i 에 대해

for j from i to $N - 1$: 가능한 모든 대여 종료일 j 에 대해

a. $avg_{ij} \leftarrow \frac{sum(cost_{ij})}{j-i+1}$: 평균 대여 비용을 계산

b. if $j - i + 1 \geq L$: 조건을 만족하면

$min_cost \leftarrow \min(min_cost, avg_{ij})$: 최소 평균 대여 비용을 업데이트

$j \geq i$



Lec. 01. 알고리즘과 문제해결

Step 1. 각 테스트 케이스별로 입력과 출력을 처리한다.

...

P.1.1. 록 페스티벌 (ID: FESTIVAL): (1) 무식하게 풀기

...

입력의 첫 줄에 주어지는 테스트 케이스의 수를 읽는다.

C = int(input())

각 테스트 케이스별로 문제를 풀고, 해답을 출력한다.

for _ in range(C):

테스트 케이스의 첫 줄에 주어지는 N과 L을 읽는다.

N, L = map(int, input().split())

다음 줄에 주어지는 날짜별 공연장 대여 비용을 읽는다.

cost = list(map(int, input().split()))

주어진 테스트 케이스에 대해 해답을 구해 출력한다.

answer = festival(N, L, cost)

print(answer)

→ 함수 이름 짓기가 귀찮을 땐, solve()라고 하자.



Lec. 01. 알고리즘과 문제해결

Step 2. 문제해결을 위한 알고리즘을 구현한다.

평균 대여 비용의 최솟값을 반환한다.

```
def festival(N, L, cost):
```

최소 평균대여비용을 무한으로 초기화한다.

```
min_cost = float('inf')
```

```
for i in range(N): # 모든 대여 시작일에 대해
```

```
    for j in range(i, N): # 모든 대여 종료일에 대해
```

```
        if j - i + 1 >= L: # 조건을 만족하면
```

평균 대여 비용을 계산한다.

```
        avg_cost = sum(cost[i:j+1]) / (j - i + 1)
```

최소 평균대여비용의 값을 갱신한다.

```
        min_cost = min(min_cost, avg_cost)
```

최소 평균대여비용의 값을 반환한다.

```
return min_cost
```

tot_cost



Lec. 01. 알고리즘과 문제해결

4단계: 어떻게 풀었는지 돌아보고, 개선할 방법이 있는지 찾아본다.

1. FESTIVAL 알고리즘의 정확성을 증명하시오.

정확성 증명을 위해 어떤 증명 방법을 선택하면 좋을까?
선택한 증명 방법으로 알고리즘이 정확함을 증명하시오.

2. FESTIVAL 알고리즘의 효율성을 평가하시오.

이 알고리즘의 시간 복잡도는 $O(n^2)$ 인가, $O(n^3)$ 인가?
이 알고리즘의 시간 복잡도를 개선할 수 있는가?

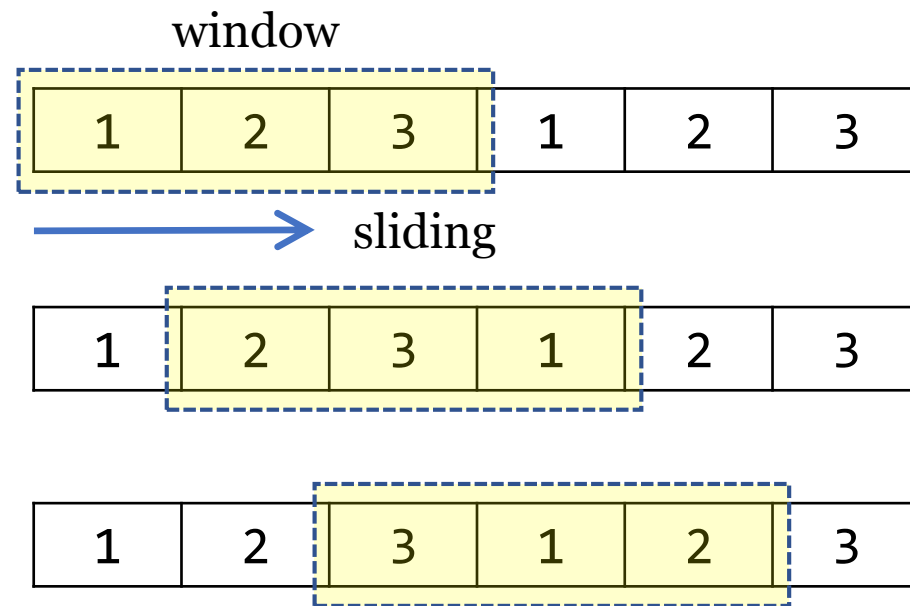


Lec. 01. 알고리즘과 문제해결

슬라이딩 윈도우: *Sliding Window*

배열이나 리스트와 같은 선형 자료구조에서 **부분 배열을 처리하는** 테크닉

윈도우(부분 배열)를 **한 번에 한 원소씩 이동하며 필요한 계산을 수행하여**
불필요한 중복 계산을 피하고, 시간 복잡도를 줄일 수 있다.





슬라이딩 윈도우 테크닉의 활용

슬라이딩 윈도우를 이용한 페스티벌 문제해결

```
def festival(N, L, cost):  
    min_cost = float('inf')  
    for i in range(N):  
        tot_cost = 0 # 부분 배열의 합 초기화  
        for j in range(i, N):  
            tot_cost += cost[j] # 부분 배열의 합 계산  
            if j - i + 1 >= L:  
                avg_cost = tot_cost / (j - i + 1) # 불필요한 중복 계산 제거  
                min_cost = min(min_cost, avg_cost)  
    return min_cost
```



Lec. 01. 알고리즘과 문제해결

누적 합: *Prefix Sum*

배열이나 리스트의 구간 합을 빠르게 계산하기 위해 미리 누적 합을 전처리하는 테크닉

	[0]	[1]	[2]	[3]	[4]	[5]
$cost =$	1	2	3	1	2	3
		i			j	

$$sum(i, j) = sum(cost[i:j + 1])$$

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
$prefix\ sum =$	1	3	6	7	9	12	0
		i			j		$[-1]$

$$sum(i, j) = psum[j] - psum[i - 1]$$

파이썬에서는 마지막 원소의 인덱스가 -1이므로

편리한 계산을 위해 누적 합의 오른쪽에 0의 값을 추가





누적 합 테크닉의 활용

누적 합을 이용한 페스티벌 문제해결

```
def festival(N, L, cost):
```

psum에 cost의 누적 합을 미리 계산해 둔다.

*shallow
copy*

```
    psum = cost[:] + [0] # 편리한 계산을 위해 마지막(-1)에 0을 추가
```

```
    for i in range(N):
```

```
        psum[i] += psum[i - 1]
```

$psum(i) = psum(i - 1) + cost[i]$

```
    min_cost = float('inf')
```

```
    for i in range(N):
```

```
        for j in range(i, N):
```

```
            if j - i + 1 >= L:
```

```
                avg_cost = (psum[j] - psum[i - 1]) / (j - i + 1)
```

```
                min_cost = min(min_cost, avg_cost)
```

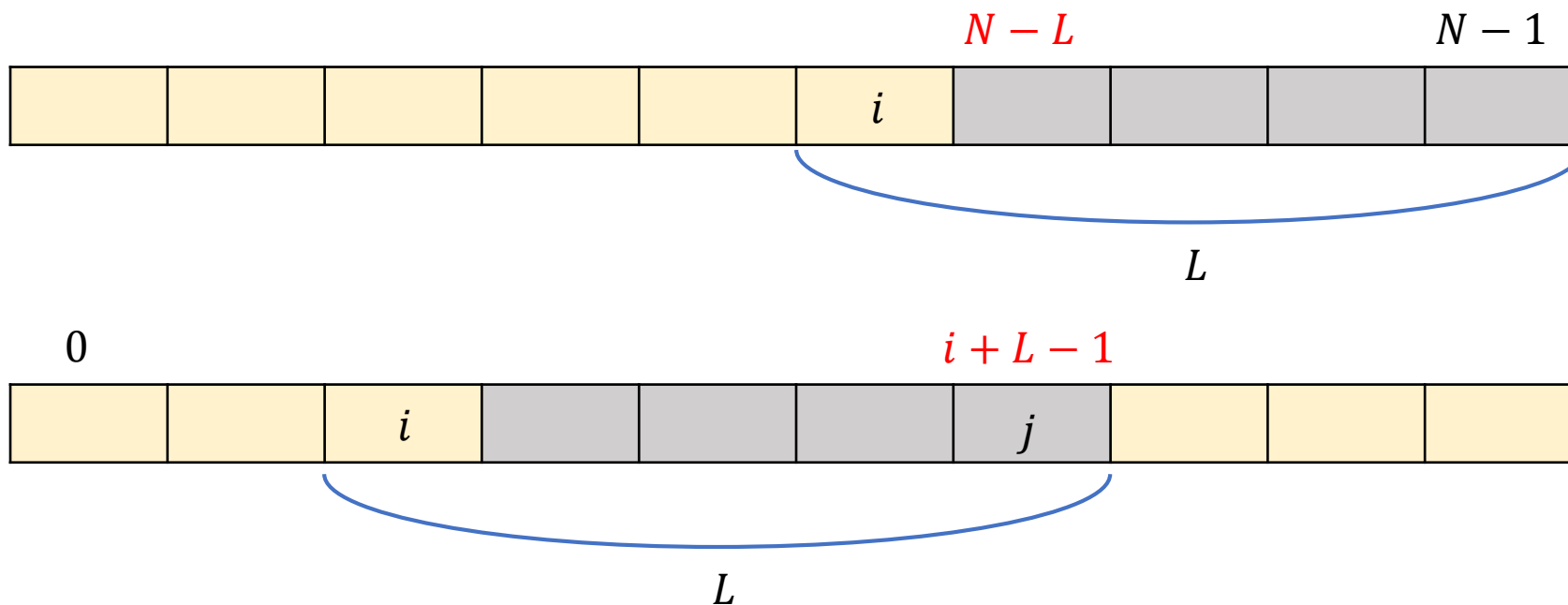
```
    return min_cost
```

누적 합을 이용하여 *tot_cost*를 바로 계산



더 개선할 방법이 있을까?

모든 (i, j) 의 쌍에 대해서 전부 살펴보면서 제약조건을 확인할 필요 없이
남은 길이가 L 보다 작으면 중단하고, 길이가 L 미만인 경우는 건너뛰도록 한다.





범위를 제한하는 테크닉의 활용

누적 합 + 범위를 제한한 페스티벌 문제해결

```
def festival(N, L, cost):  
    psum = cost[:] + [0]  
    for i in range(N):  
        psum[i] += psum[i - 1]  
    min_cost = float('inf')  
    for i in range(N - L + 1): # 남은 길이가 L보다 작으면 중단  
        for j in range(i + L - 1, N): # 길이가 L 이상인 지점에서 시작  
            avg_cost = (psum[j] - psum[i - 1]) / (j - i + 1)  
            min_cost = min(min_cost, avg_cost)  
    return min_cost
```

이 알고리즘의 시간 복잡도는?



아직도 더 개선할 방법이 있을까?

빠른 입출력 사용: input() 함수 대신에 sys.stdin 모듈의 readline() 함수를 사용해본다.

```
# input() 함수 대신에 sys.stdin 모듈의 readline() 함수 사용
```

```
import sys
```

```
input = sys.stdin.readline
```

```
for _ in range(int(input())):
```

```
    N, L = map(int, input().split())
```

```
    cost = list(map(int, input().split()))
```

```
    print(festival(N, L, cost))
```

파이썬에서는 함수도 일급 객체(*First-Class Object*):

1. 변수에 할당할 수 있다.
2. 함수의 인자로 사용할 수 있다.
3. 함수의 반환값으로 사용할 수 있다.



AL2.1.2: 최대 연속 부분합 찾기 (ID: MAXSUM)

문제 N 개의 정수를 담고 있는 배열 A 가 주어졌을 때, 여기서 가능한 연속 부분합 중에서 가장 큰 연속 부분합을 구하는 프로그램을 작성하라.

여기서 연속 합이라는 것은 배열 $A = \{a_1, a_2, \dots, a_n\}$ 에서 아무 값도 선택을 하지 않거나 (이 경우 합은 0), 배열의 임의의 i 번째 수 부터 j 번째 수 까지($a_i, a_{i+1}, \dots, a_j$) ($1 \leq i \leq j \leq N$)를 합한 값을 뜻한다.



입력

입력의 첫번째 줄에는 테스트 케이스의 개수 T 가 입력된다.

그리고 그 다음줄 부터 한줄에 하나씩 T 개의 테스트 케이스가 입력된다.

테스트 케이스의 첫번째 줄에는 정수 N ($1 \leq N \leq 105$)가 입력된다.

그리고 그 다음줄에는 N 개의 배열에 담긴 숫자가 순서대로 입력된다.

숫자의 범위는 -100 이상 100 이하의 정수다.

출력

한줄에 하나씩 테스트 케이스의 순서대로 각 테스트케이스에 대한 가장 큰 연속 부분합을 출력한다.



Lec. 01. 알고리즘과 문제해결

예제 입력

```
2
4
1 2 3 4
3
-1 0 1
```

예제 출력

```
10
1
```



최대 연속 부분합 문제: *maximum subarray problem*

1. 문제를 읽고 이해하시오.
2. 어떻게 풀 지 계획을 세우시오.
누적 합을 이용해서 문제를 풀 수 있을까?
3. 계획을 수행해서 문제를 해결하시오.
4. 어떻게 풀었는지 돌아보고, 개선할 방법이 있는지 찾아보시오.
이 문제를 $O(N \lg N)$ 으로 풀 수 있을까?
이 문제를 $O(N)$ 으로 풀 수 있을까?

※ 교재 pp. 117-121 참고



Lec. 01. 알고리즘과 문제해결

최대 연속 부분합 문제를 **누적 합**으로 풀기

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
$A =$	-2	1	-3	4	-1	2	1	-5	4
				i			j		

최대 연속 부분합은 $4 + (-1) + 2 + 1 = 6$

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[-1]
$prefix\ sum =$	-2	-1	-4	0	-1	1	2	-3	1	0
				i			j			

최대 연속 부분합은 $2 - (-4) = 6$



Lec. 01. 알고리즘과 문제해결

'''

P.1.2. 최대 연속 부분합 찾기 (ID: MAXSUM): (1) 누적 합

'''

```
for _ in range(int(input())):  
    N = int(input()) # 정수 배열의 크기  
    A = list(map(int, input().split())) # 정수 배열의 원소  
    print(maxsum(N, A))
```



Lec. 01. 알고리즘과 문제해결

누적 합을 이용한 최대 연속 부분합 찾기

```
def maxsum(n, A):
```

```
    # A의 누적 합을 계산한다.
```

```
    psum = A[:] + [0]
```

```
    for i in range(n):
```

```
        psum[i] += psum[i - 1]
```

```
    # 최대 연속 부분합을 찾는다.
```

```
    maxs = 0 # 최대 연속 부분 합
```

```
    for i in range(n):
```

```
        for j in range(i, n):
```

```
            curs = psum[j] - psum[i - 1] # [i, j] 구간의 부분 합
```

```
            maxs = max(maxs, curs) # 최대 부분 합 업데이트
```

```
    return maxs
```

이 알고리즘의 시간 복잡도는? $O(n^2)$



이 문제를 $O(N \lg N)$ 으로 풀 수 있을까?

분할 정복법을 이용하여 해결한다.

주어진 배열 A 를 절반으로 나누면 두 개의 더 작은 부분 배열로 나누어 정복할 수 있다.

1. 최대 부분 합의 구간은 왼쪽 부분 배열에 포함되어 있을 수 있다.
2. 최대 부분 합의 구간은 오른쪽 부분 배열에 포함되어 있을 수 있다.
3. 최대 부분 합의 구간이 왼쪽과 오른쪽 부분 배열에 걸쳐 있을 수 있다.



1번의 경우



2번의 경우



3번의 경우



Lec. 01. 알고리즘과 문제해결

'''

P.1.2. 최대 연속 부분합 찾기 (ID: MAXSUM): (2) 분할 정복

'''

```
import sys
```

```
input = sys.stdin.readline
```

```
INF = float('inf')
```

```
for _ in range(int(input())):
```

```
    N = int(input()) # 정수 배열의 크기
```

```
    A = list(map(int, input().split())) # 정수 배열의 원소
```

```
    print(maxsum(0, N - 1))
```

전체 구간 $[0, N - 1]$ 에서의 최댓값을 출력

함수나 클래스의 내부가 아닌 경우, 전역 변수로 생성된다.



최대 연속 부분합 문제를 분할정복으로 풀기

```
def maxsum(low, high):
```

```
    if low == high: # 종료 조건: 구간의 길이가 1인 경우
```

```
        return max(A[low], 0)
```

```
    else:
```

```
        mid = (low + high) // 2 # 배열을 절반으로 분할
```

```
        lmax = maxsum(low, mid) # 1. 왼쪽 부분 배열의 최대 부분 합
```

```
        rmax = maxsum(mid + 1, high) # 2. 오른쪽 부분 배열의 최대 부분 합
```

```
        cmax = maxsum_crossing(low, mid, high) # 3. 양쪽에 걸쳐 있는 최대 부분 합
```

```
    return max(lmax, rmax, cmax) # 셋 중의 최댓값을 반환
```



Lec. 01. 알고리즘과 문제해결

양쪽에 걸쳐 있는 최대 부분합을 반환한다.

```
def maxsum_crossing(low, mid, high):
```

```
    # A[mid]에서 끝나는 최대 부분 합을 찾는다.
```

```
    left = -INF; curs = 0
```

```
    for i in range(mid, low - 1, -1):
```

```
        curs += A[i]
```

```
        left = max(left, curs)
```

```
    # A[mid+1]에서 시작하는 최대 부분 합을 찾는다.
```

```
    right = -INF; curs = 0
```

```
    for j in range(mid + 1, high + 1):
```

```
        curs += A[j]
```

```
        right = max(right, curs)
```

```
    # left와 right를 더하면 양쪽에 걸친 최대 부분 합
```

```
    return left + right
```

※ P.3.2 울타리 잘라내기 문제를
풀어 본 이후에 다시 풀어볼 것.



이 문제를 $O(N)$ 으로 풀 수 있을까?

동적 계획법을 이용하여 해결한다. (*Kadane's* Algorithm)

$maxAt(i)$: $A[i]$ 를 오른쪽 끝으로 갖는 구간의 최대 합

$A[i]$ 에서 끝나는 최대 합 부분 구간은

1. 항상 $A[i]$ 하나만으로 구성되어 있거나,
2. $A[i - 1]$ 을 오른쪽 끝으로 갖는 최대 합 부분 구간의 오른쪽에 $A[i]$ 를 붙인 형태로 구성

$$maxAt(i) = \max(0, maxAt(i - 1) + A[i])$$



Lec. 01. 알고리즘과 문제해결

최대 연속 부분합 문제를 동적 계획법으로 풀기: Kadane's algorithm

```
def maxsum(n, A):
    curs, maxs = 0, 0
    for i in range(n):
        # maxAt(i) = max(maxAt(i - 1), 0) + A[i]
        curs = max(curs, 0) + A[i]
        maxs = max(maxs, curs)
    return maxs
```

파이썬에서는 다중 할당이 가능

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
$A =$	-2	1	-3	4	-1	2	1	-5	4
$curs =$	-2	1	-2	4	3	5	6	1	5
$maxs =$	0	1	1	4	4	5	6	6	6

Any Questions?

